

TOAE209/TOAH209 – Design and Analysis of Algorithms

Week 15: Practical Exercises

Foreign Trade University

Instructions

For each problem below there is a starter file `exercises/starter_code_problemNN.py` containing function signatures with `raise NotImplementedError` bodies, and a small `__main__` block with tests. Implement the missing functions so the tests pass. A reference solution is provided in `solutions/solution_problemNN.py` – try the problem yourself before looking!

All problems use only the Python 3.10+ standard library. Shared helper functions live in `starter_code.py` at the week root: `make_rng`, `generate_array`, `generate_distinct_array`, `generate_stream`, `generate_graph`, `cut_size`, `is_prime_trial_division`.

Determinism is mandatory. Every randomized routine must take a `seed` and route all randomness through `random.Random(seed)` (never the global `random` module). All tests seed their generators and assert on *loose* statistical bounds, so a correct implementation passes every time and never flakes.

The 12 problems are grouped into three themes:

- **Randomized Sorting & Selection** (Problems 1–3): Fisher–Yates shuffle, randomized QuickSort, randomized Quickselect.
- **Sampling & Simulation** (Problems 4–5, 8–9): reservoir sampling, contention resolution, Monte Carlo π , and Las Vegas vs. Monte Carlo.
- **Randomized Graph & Number Algorithms** (Problems 6–7, 10–12): Karger’s min-cut and amplification, Miller–Rabin primality, min-cut success-probability checks, and universal hashing.

1 Randomized Sorting and Selection

Problem 1 – Fisher–Yates Shuffle

Implement `fisher_yates_shuffle(arr, seed)` that returns a uniformly random permutation of `arr` using the in-place Fisher–Yates (Knuth) shuffle: for i from $n - 1$ down to 1, pick j uniformly in $[0, i]$ and swap positions i and j . Route all randomness through `random.Random(seed)`.

Verify (deterministically): the result is always a permutation of the input; and over many seeded trials, the empirical distribution of the first element (or of a small permutation) is roughly uniform (a loose chi-square-style spread check).

Problem 2 – Randomized QuickSort

Implement `randomized_quicksort(arr, seed)` returning a sorted copy, choosing each pivot uniformly at random (seeded). Also return, via `quicksort_with_comparisons(arr, seed)`, the number of element comparisons performed.

Verify: the output equals `sorted(arr)` on many seeded instances; and the comparison count stays within a generous constant factor of $n \log_2 n$ (e.g. $\leq 10 n \log_2 n$ averaged over trials), confirming the expected $O(n \log n)$ behaviour.

Problem 3 – Randomized Quickselect

Implement `randomized_quickselect(arr, k, seed)` returning the k -th smallest element (1-indexed) using random pivots (seeded), recursing into only the relevant side.

Verify: for many seeded instances and every k , the result equals `sorted(arr)[k-1]`.

2 Sampling and Simulation

Problem 4 – Reservoir Sampling

Implement `reservoir_sample(stream, k, seed)` returning a uniform random k -subset of `stream` (any iterable of unknown length) in a single pass using $O(k)$ space: keep the first k items, and for the t -th item ($t > k$) accept it with probability k/t , replacing a uniformly random reservoir slot.

Verify: over many seeded trials on a stream of n items, each element’s empirical inclusion frequency is close to k/n (a loose absolute-tolerance bound).

Problem 5 – Contention Resolution Simulation

Implement `simulate_contention(n, seed)`: simulate n processes that each attempt a shared resource with probability $p = 1/n$ per round; a process succeeds in a round iff it is the only one attempting. Return the number of rounds until *all* n have succeeded at least once. Then implement `average_rounds(n, num_trials, seed)` averaging this over seeded trials.

Verify: the average number of rounds grows like $\Theta(n \log n)$ – specifically, it lies within a loose constant-factor window of $n \ln n$ for a couple of sizes.

Problem 8 – Monte Carlo Estimation of π

Implement `estimate_pi(num_samples, seed)`: throw `num_samples` random points uniformly into $[0, 1]^2$ (seeded) and return $4 \times$ the fraction landing inside the unit quarter-circle ($x^2 + y^2 \leq 1$).

Verify: for large `num_samples` the estimate is within a loose tolerance of π (e.g. $|\hat{\pi} - \pi| < 0.1$ at 2×10^5 samples with a fixed seed).

Problem 9 – Las Vegas vs. Monte Carlo

For the task “find an index of `target` in a list known to contain it,” implement: `las_vegas_find(arr, target, seed)` that repeatedly probes random indices until it hits `target` (always correct, random time); and `monte_carlo_find(arr, target, max_probes, seed)` that probes at most `max_probes` random indices and returns an index or `None` (fixed time, may fail).

Verify: the Las Vegas variant is *always* correct over many seeded trials (returns an index i with `arr[i] == target`); the Monte Carlo variant runs a bounded number of probes and may sometimes return `None` – but is never *wrong* when it returns an index.

3 Randomized Graph and Number Algorithms

Problem 6 – Karger’s Contraction (Single Run)

Implement `karger_min_cut(graph, seed)` performing one run of Karger’s contraction: repeatedly contract a uniformly random edge (merging endpoints, keeping parallel edges, dropping self-loops) until two super-vertices remain; return the number of edges between them (the cut size of this run).

Verify: on small connected graphs the returned value is always $\geq$ the true global min-cut and is a valid cut size; runs with different seeds may differ.

Problem 7 – Karger with Amplification

Implement `karger_min_cut_amplified(graph, trials, seed)` that runs Problem 6 `trials` times (with derived seeds) and returns the smallest cut found. Also provide `brute_force_min_cut(graph)` computing the true global min-cut by trying all vertex bipartitions (for small n).

Verify: with enough trials, the amplified result equals the brute-force min-cut on several small graphs (high probability; the seeds are fixed so the test is deterministic).

Problem 10 – Miller–Rabin Primality Test

Implement `miller_rabin(n, seed, rounds=20)`: the Miller–Rabin probabilistic test with `rounds` random witnesses (seeded). Handle small/even cases directly.

Verify: against `is_prime_trial_division` (from `starter_code.py`) over a range of integers (e.g. $0 \dots 500$), the two agree on every value with a fixed seed and `rounds=20`.

Problem 11 – Min-Cut Success-Probability Empirical Check

Implement `single_run_success_rate(graph, num_runs, seed)`: run Problem 6 `num_runs` times (derived seeds), and return the fraction of runs whose cut equals the true global min-cut (use `brute_force_min_cut` from Problem 7).

Verify: on a small graph the observed success fraction is at least the theoretical floor $1/\binom{n}{2}$ (with a comfortable margin and enough runs, deterministically seeded).

Problem 12 – Universal-Hashing Collision Count

Implement `universal_hash_family(p, m, seed)` returning a hash function $h(x) = ((ax + b) \bmod p) \bmod m$ with random $a \in \{1, \dots, p-1\}, b \in \{0, \dots, p-1\}$ (seeded). Implement `count_collisions(keys, h)` counting colliding *pairs*, and `average_collisions(n, m, num_trials, seed)` averaging the collision count over seeded random hash functions on n distinct keys.

Verify: the average number of colliding pairs is near $\binom{n}{2}/m \approx n^2/(2m)$ over the trials (a loose relative-tolerance bound).

4 LeetCode Practice

After completing the problems above, practise this week's randomized techniques (Fisher–Yates, reservoir sampling, randomized quickselect, rejection sampling) on the following [LeetCode](#) problems. Difficulty is LeetCode's own label.

- [Shuffle an Array](#) – *Medium* – Fisher–Yates shuffle.
- [Random Pick with Weight](#) – *Medium* – prefix sums + binary search.
- [Random Pick Index](#) – *Medium* – reservoir sampling.
- [Linked List Random Node](#) – *Medium* – reservoir sampling.
- [Insert Delete GetRandom O\(1\)](#) – *Medium* – randomized data structure.
- [Implement Rand10\(\) Using Rand7\(\)](#) – *Medium* – rejection sampling.
- [Kth Largest Element in an Array](#) – *Medium* – randomized quickselect.
- [Random Point in Non-overlapping Rectangles](#) – *Medium* – weighted random sampling.